

XGBoost: Comprehensive Introduction and Technical Deep Dive

Papers

- <https://browse.arxiv.org/pdf/1603.02754.pdf>

1. Introduction to XGBoost

What is XGBoost?

XGBoost (eXtreme Gradient Boosting) is **NOT** a machine learning algorithm itself, but rather a **highly optimized library** built on top of the existing Gradient Boosting algorithm. It was created by **Tianqi Chen** in 2014.



Key Insight

XGBoost = Gradient Boosting + Software Engineering Optimizations

The library merges:

- Machine Learning concepts (from Gradient Boosting)
- Advanced Software Engineering techniques

This combination makes it a "super powerful" machine learning library.

2. Historical Context and Evolution

Why Gradient Boosting Was Chosen as Foundation?

Feature	Benefit	Impact
Flexible	Can use any differentiable loss function	Works on regression, classification, ranking, custom problems
Performance	Consistently good results across datasets	High accuracy and reliability
Robust	With proper regularization, avoids overfitting	Handles missing values internally

Feature	Benefit	Impact
Kaggle-proven	Already popular in competitions	Real-world validation

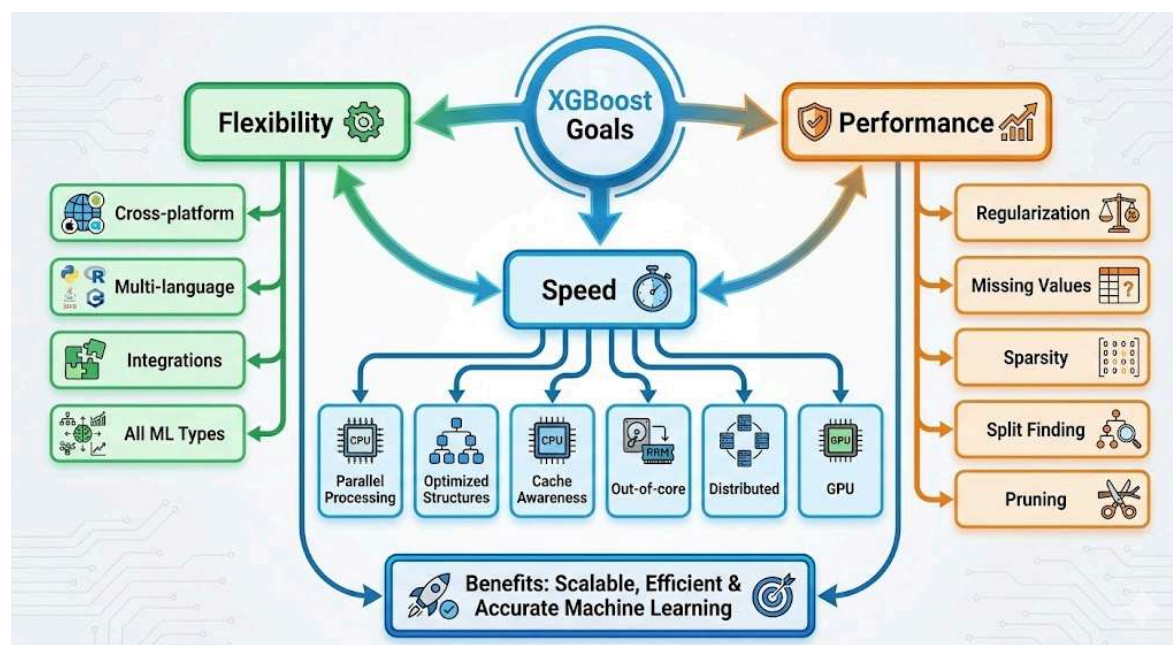
The "Extreme" in XGBoost

The name "eXtreme" comes from the **extreme software optimizations** applied:

- Parallel processing
- Cache-aware algorithms
- Out-of-core computing
- Distributed computing
- GPU support
- Advanced data structures

3. Core Philosophy and Design Principles

Tianqi Chen focused on **three major areas** while building XGBoost:



4. XGBoost Features: Flexibility

4.1 Cross-Platform Support

XGBoost models can run on **any operating system**:

- Windows
- Linux
- macOS

Key Feature: A model trained on Linux can be loaded and used on Windows or macOS without issues.

4.2 Multiple Programming Language Support

Unlike most ML libraries that support only 1-2 languages, XGBoost provides **wrappers for multiple languages**:

Language	Support	Use Case
Python	✓ Full	Data science, research
R	✓ Full	Statistical analysis
Java	✓ Full	Enterprise applications
Scala	✓ Full	Big data processing
Ruby	✓ Full	Web applications
Swift	✓ Full	iOS applications
Julia	✓ Full	Scientific computing
C/C++	✓ Full	Production systems

Cross-Language Model Loading Example

Python - Train and Save:

```
import xgboost as xgb

# Train model
model = xgb.XGBClassifier()
model.fit(X_train, y_train)

# Save model
model.save_model('model.json')
```



Java - Load and Predict:


```
import ml.dmlc.xgboost4j.java.Booster;
import ml.dmlc.xgboost4j.java.XGBoost;

// Load model trained in Python
Booster booster = XGBoost.loadModel("model.json");

// Make predictions
float[][] predictions = booster.predict(testData);
```



4.4 Support for All ML Problem Types

XGBoost handles diverse machine learning tasks:

Problem Type	XGBoost Capability	Example Use Case
Regression	Linear regression objectives	House price prediction
Binary Classification	Logistic objectives	Spam detection
Multi-class Classification	Softmax objectives	Image classification
Ranking	Ranking objectives	Search results ordering
Time Series	Custom objectives	Stock price forecasting
Anomaly Detection	One-class objectives	Fraud detection
Custom Problems	User-defined loss functions	Domain-specific tasks

Custom Loss Function Support

You can define any differentiable loss function:

```
import xgboost as xgb
import numpy as np

# Custom loss function that penalizes false negatives more
def custom_loss(y_pred, y_true):
    # Gradients
    grad = np.where(y_true == 1,
                    -2 * (y_true - y_pred), # Higher weight for positives
                    -(y_true - y_pred))

    # Hessians
    hess = np.where(y_true == 1, 2.0, 1.0)
```



```
    return grad, hess

# Use custom objective
model = xgb.XGBClassifier(objective=custom_loss)
```

5. XGBoost Features: Speed Optimizations {#speed}

Empirical Speed Comparison

Test Setup:

- Dataset: 10,000 rows × 200 columns (synthetic data)
- Comparison: Gradient Boosting vs XGBoost

Results:

Gradient Boosting Training Time: 72 seconds
XGBoost Training Time: 6 seconds
Speedup: 12× faster



For large datasets (millions of rows), the speedup can be even more dramatic (often 10-15×).

5.1 Parallel Processing

How Decision Trees are Built

Standard tree building process:

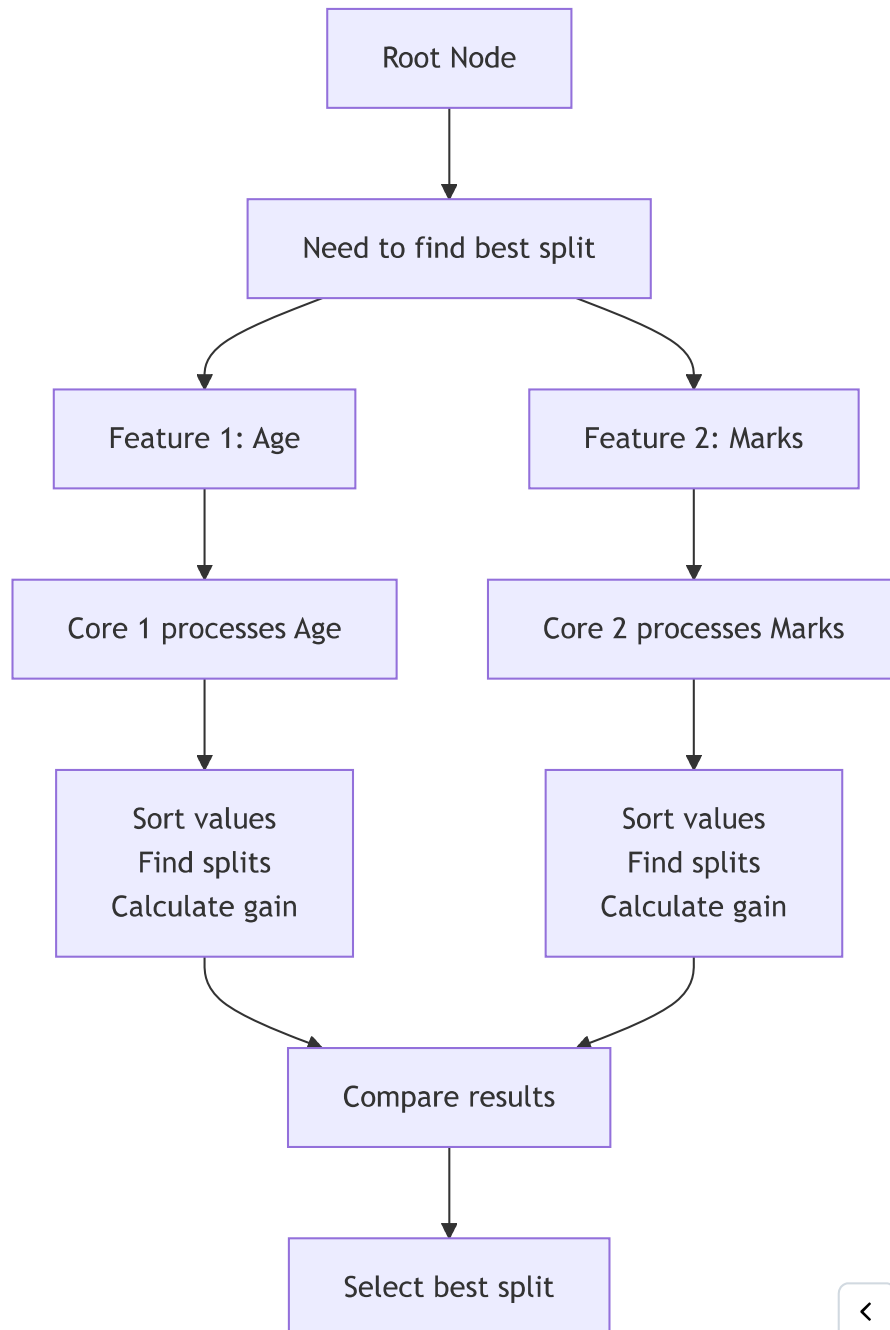
1. Sort feature values
2. Find splitting points
3. Calculate gain for each split
4. Select best split
5. Repeat for each node

The Misconception

Question: "Boosting is sequential (one tree after another), so how can XGBoost use parallel processing?"

Answer: The parallelization is **not across trees**, but **within each tree** during construction.

Parallel Split Finding



Example with Sample Data:

Age	Marks	Placement (Lakhs)
21	98	13
19	89	11
17	95	15

Sequential Processing (Old Way):

Time = Process Age + Process Marks
= $t_1 + t_2$



Parallel Processing (XGBoost):

Time = $\max(\text{Process Age}, \text{Process Marks})$
= $\max(t_1, t_2)$



For 200 features with 8 CPU cores:

Sequential: $200 \times t$
Parallel (8 cores): $200/8 \times t = 25 \times t$
Speedup: 8×



Activation:

```
model = xgb.XGBClassifier(n_jobs=-1) # Use all cores
```



5.2 Optimized Data Structures

Column Block Format

Traditional (Row-wise) Storage:

Block 1: [CGPA=8.5, IQ=120, Placement=13L]
Block 2: [CGPA=7.2, IQ=115, Placement=11L]
Block 3: [CGPA=9.1, IQ=130, Placement=15L]

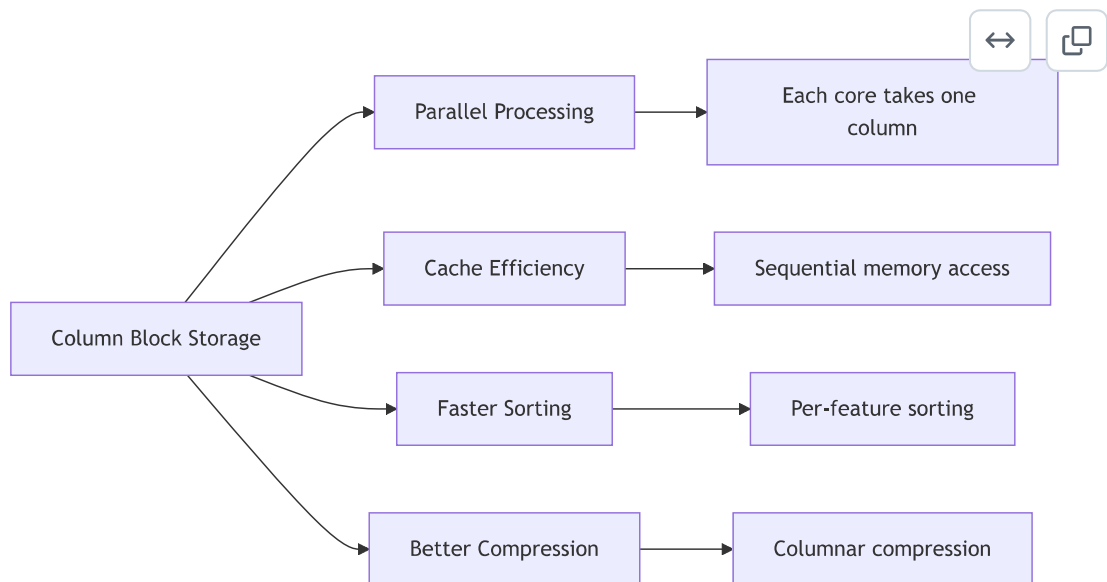


XGBoost (Column-wise) Storage:

Column Block 1 (CGPA): [8.5, 7.2, 9.1]
 Column Block 2 (IQ): [120, 115, 130]
 Column Block 3 (Placement): [13L, 11L, 15L]



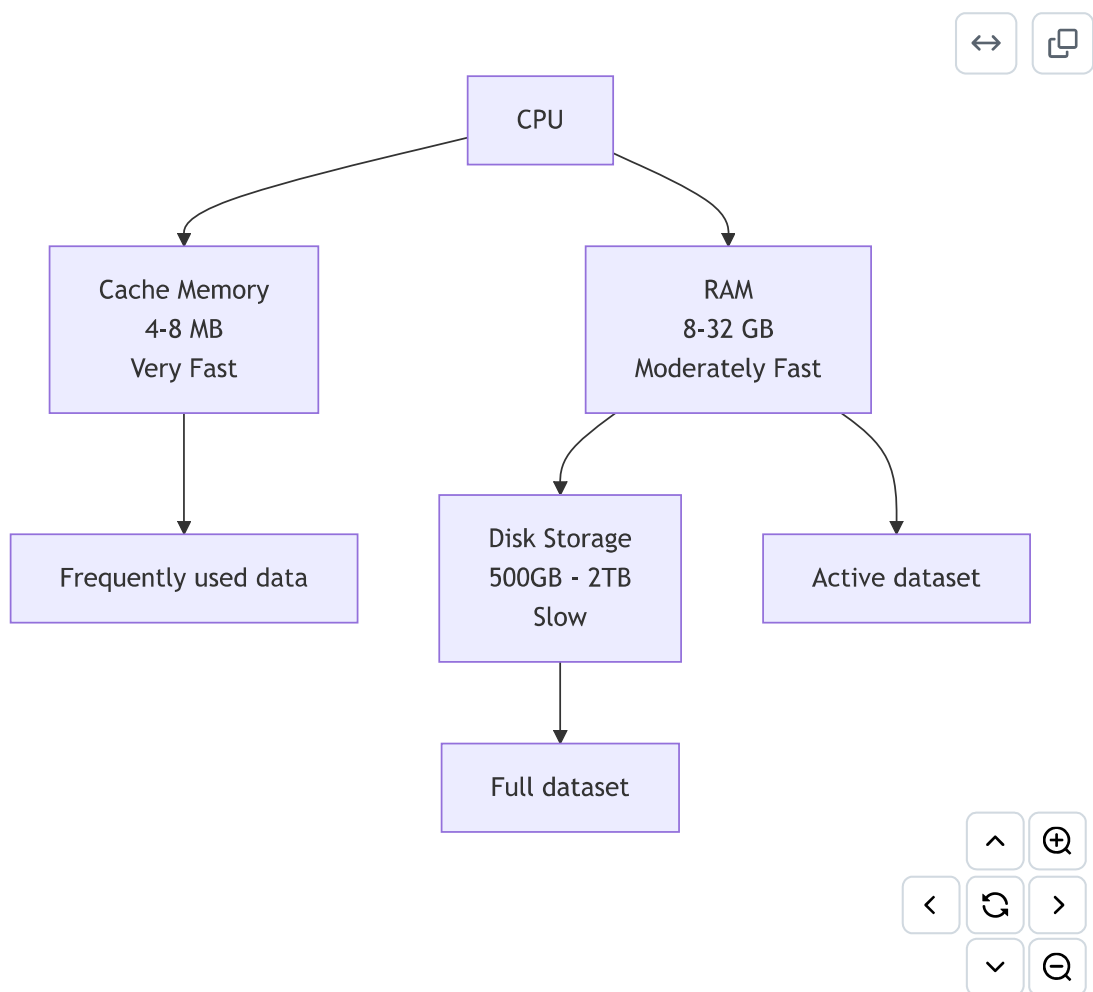
Benefits of Column Blocks



Aspect	Row-wise	Column-wise (XGBoost)
Parallel access	Difficult	Easy
Cache utilization	Poor	Excellent
Memory layout	Scattered	Contiguous
Split finding	Slow	Fast

5.3 Cache Awareness

Memory Hierarchy



Cache-Aware Algorithm

Histogram-Based Training:

XGBoost uses histograms for continuous features:

Original values: [50, 55, 60, 65, 70, 75, 80, 85, 90, 95]

Histogram bins:

Bin 1: 50-60 (contains 50, 55, 60)

Bin 2: 60-70 (contains 65, 70)

Bin 3: 70-80 (contains 75, 80)

Bin 4: 80-90 (contains 85, 90)

Bin 5: 90-100 (contains 95)

Key Optimization:

- Bin boundaries stored in **cache memory**

- Frequently accessed during tree building
- Eliminates RAM access overhead

Kitchen Analogy

Scenario: Cooking in kitchen



RAM = Refrigerator (in another room)

Cache = Counter space (next to you)

Without cache awareness:

- Go to fridge for salt
- Go to fridge for pepper
- Go to fridge for oil
- (Multiple trips, time wasted)

With cache awareness:

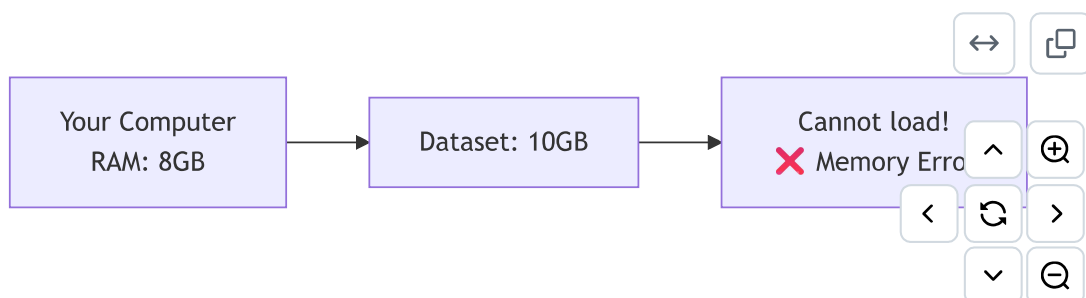
- Bring salt, pepper, oil to counter
- Use directly while cooking
- (Single trip, faster cooking)

Performance Impact:

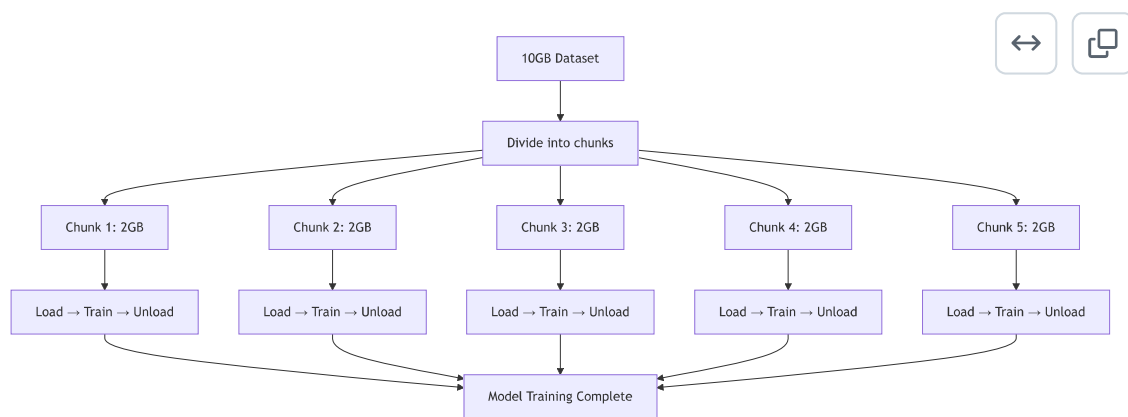
- Cache hit: ~1-4 CPU cycles
- RAM access: ~100-200 CPU cycles
- Speedup: 50-200× for cached data

5.4 Out-of-Core Computing

The Problem



The Solution: Data Chunking



Implementation

```

import xgboost as xgb

# Enable out-of-core computing
params = {
    'tree_method': 'hist', # Activates out-of-core
    'max_bin': 256
}

model = xgb.train(params, dtrain, num_boost_round=100)
  
```

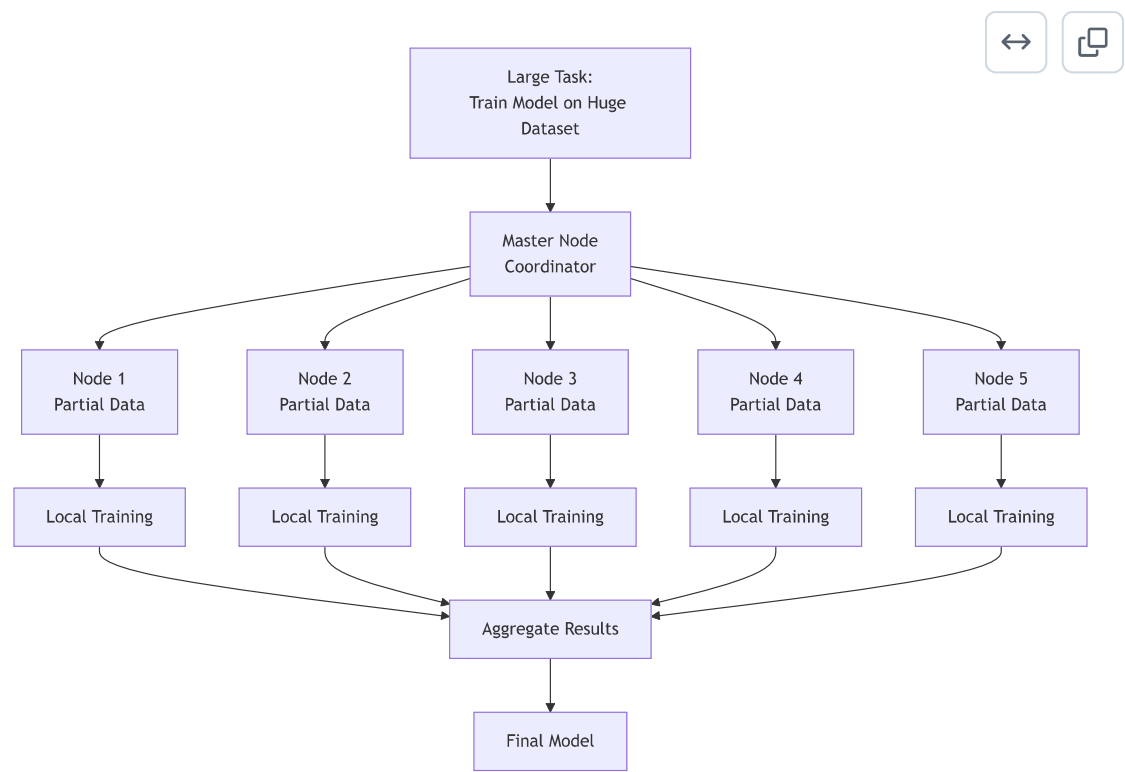
Process:

1. Dataset divided into chunks
2. Each chunk loaded sequentially
3. Model trained incrementally
4. Chunk unloaded after processing
5. Next chunk loaded

Works together with **cache memory** to store important information across chunks.

5.5 Distributed Computing

Concept



Out-of-Core vs Distributed Computing

Aspect	Out-of-Core	Distributed
Machines	1 machine	Multiple machines
Processing	Sequential chunks	Parallel nodes
Speed	Limited by single CPU	Much faster (multiple CPUs)

Aspect	Out-of-Core	Distributed
Use Case	Dataset > RAM	Dataset >> RAM or need speed
Data Flow	Chunk 1 → Chunk 2 → Chunk 3	All chunks processed simultaneously

Distributed Training Process

Step 1: Data Partitioning

10GB Dataset → Split into 5 parts



Node 1: D1 (2GB)

Node 2: D2 (2GB)

Node 3: D3 (2GB)

Node 4: D4 (2GB)

Node 5: D5 (2GB)

Step 2: Parallel Training

Each node finds best split for its data:



Feature: Age

Node 1 → Split: Age > 25 (Gain: 0.45)

Node 2 → Split: Age > 30 (Gain: 0.52)

Node 3 → Split: Age > 28 (Gain: 0.48)

Node 4 → Split: Age > 32 (Gain: 0.50)

Node 5 → Split: Age > 27 (Gain: 0.46)

Feature: Marks

Node 1 → Split: Marks > 75 (Gain: 0.60)

Node 2 → Split: Marks > 80 (Gain: 0.65)

Node 3 → Split: Marks > 78 (Gain: 0.62)

Node 4 → Split: Marks > 82 (Gain: 0.64)

Node 5 → Split: Marks > 77 (Gain: 0.61)

Step 3: Aggregation

Master node receives all results



Compares: Best split = Marks > 80 (Gain: 0.65)

Sends decision to all nodes

Implementation with Dask



```
import xgboost as xgb
import dask.dataframe as dd
from dask.distributed import Client

# Setup distributed cluster
client = Client()

# Load data with Dask
df = dd.read_csv('large_dataset.csv')

# Convert to DMatrix
dtrain = xgb.dask.DaskDMatrix(client, df[features], df[target])

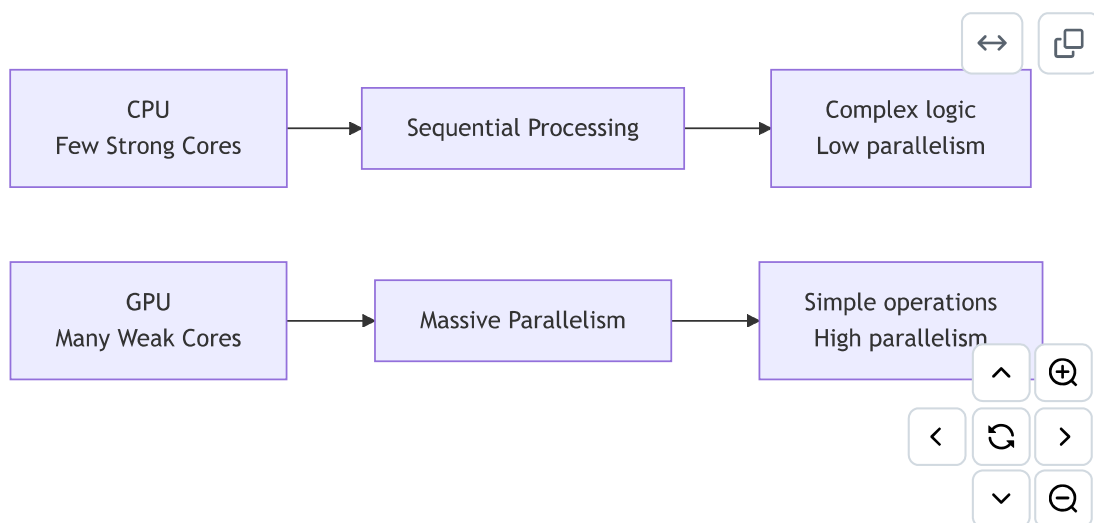
# Distributed training
output = xgb.dask.train(
    client,
    params,
    dtrain,
    num_boost_round=100
)
```

5.6 GPU Support

Why GPU for Machine Learning?

CPU vs GPU Architecture:

Component	CPU	GPU
Cores	4-16 powerful cores	1000s of smaller cores
Best for	Complex sequential tasks	Simple parallel tasks
Computation Type	General purpose	Specialized (matrix operations)

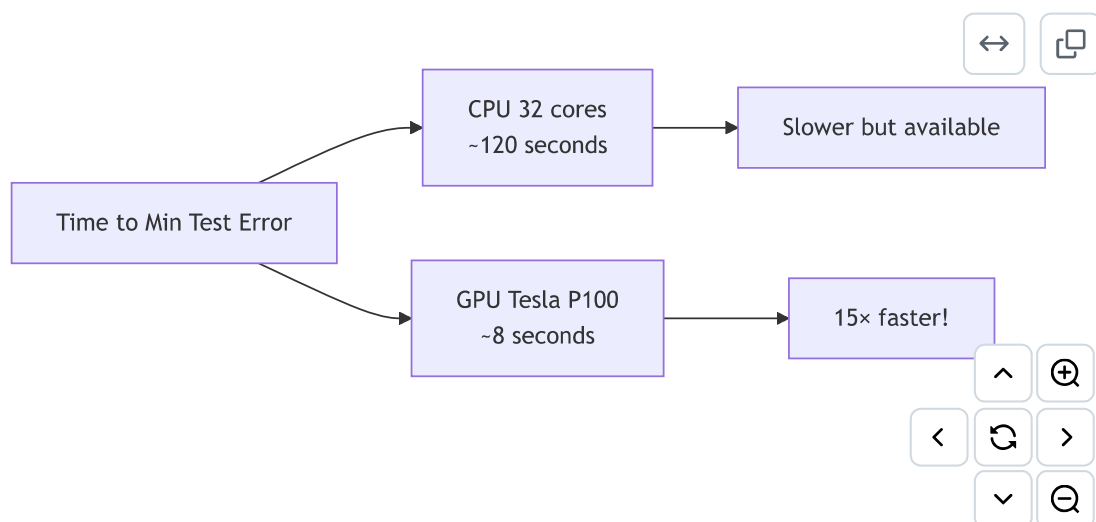


XGBoost GPU Operations

XGBoost parallelizes these operations on GPU:

1. **Histogram construction** (binning feature values)
2. **Split finding** (evaluating splits across all features)
3. **Tree growing** (building multiple levels simultaneously)

Performance Comparison



GPU Activation

```
import xgboost as xgb
```

```
params = {
```

```

    'tree_method': 'gpu_hist', # Enable GPU
    'predictor': 'gpu_predictor' # GPU prediction
}

model = xgb.train(params, dtrain, num_boost_round=100)

```

Requirements:

- CUDA-compatible GPU
- CUDA toolkit installed
- XGBoost compiled with GPU support

6. XGBoost Features: Performance Enhancements {#performance}

6.1 Regularized Learning Objective

Mathematical Formulation

The XGBoost objective function includes regularization by default:

$$L = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Where:

- $l(y_i, \hat{y}_i)$ = Loss function (differentiable)
- $\Omega(f_k)$ = Regularization term

Regularization Term:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Where:

- T = Number of leaves
- w_j = Weight of leaf j
- γ = Complexity penalty for leaves
- λ = L2 regularization parameter

Comparison with Gradient Boosting

Standard Gradient Boosting:

Minimize: $\text{Loss}(y, \hat{y})$



Regularization methods:

- Learning rate shrinkage
- Tree pruning (post-construction)

XGBoost:

Minimize: $\text{Loss}(y, \hat{y}) + \text{Regularization}(\text{tree})$



Built-in regularization:

- L1 (Lasso) - alpha parameter
- L2 (Ridge) - lambda parameter
- Gamma - leaf penalty

Benefits

Aspect	Impact
Overfitting reduction	Model generalizes better
Tree complexity control	Prevents overly complex trees
Automatic feature selection	L1 regularization zeros out weak features
Better test performance	More robust across datasets

Example

```
params = {  
    'lambda': 1.0,      # L2 regularization  
    'alpha': 0.0,      # L1 regularization  
    'gamma': 0.1,      # Minimum loss reduction for split  
}
```



6.2 Handling Missing Values

Traditional Approach

Most ML algorithms:

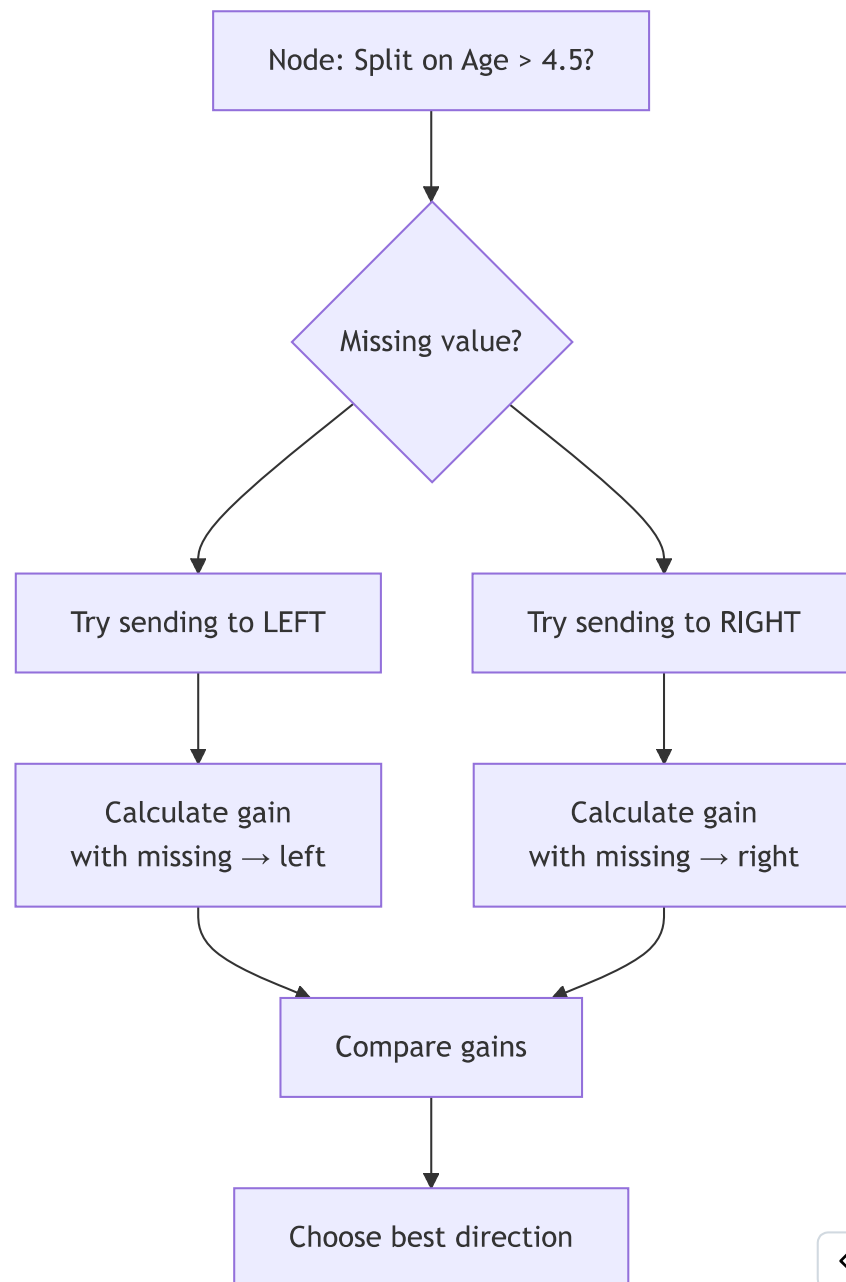
```
# Preprocessing required
df = df.dropna() # Drop rows with missing values
# OR
df = df.fillna(df.mean()) # Impute missing values
```



XGBoost Approach

No preprocessing needed! XGBoost handles missing values internally.

Sparsity-Aware Split Finding



Detailed Example

Sample Data:

Age	Marks	Placement
4	70	Yes

Age	Marks	Placement
5	85	Yes
6	90	Yes
?	75	No
8	95	Yes

Split Candidate: Age > 5.5

Test 1: Missing → LEFT

Left: [4, 5, ?] → [Yes, Yes, No] → Entropy calculation
 Right: [6, 8] → [Yes, Yes] → Entropy calculation
 Information Gain: 0.42



Test 2: Missing → RIGHT

Left: [4, 5] → [Yes, Yes] → Entropy calculation
 Right: [6, 8, ?] → [Yes, Yes, No] → Entropy calculation
 Information Gain: 0.58



Decision: Send missing values RIGHT (higher gain)

Code Demonstration

```
import xgboost as xgb
import numpy as np
import pandas as pd

# Create data WITH missing values
data = pd.DataFrame({
    'feature1': [1, 2, 3, 4, 5],
    'feature2': [10, np.nan, 30, 40, 50], # Missing value
    'target': [0, 1, 0, 1, 1]
})

# NO preprocessing needed!
X = data[['feature1', 'feature2']]
y = data['target']

# Train directly
model = xgb.XGBClassifier()
model.fit(X, y)
```



```
# Accuracy: 1.0 (model handled missing value internally)
```

6.3 Sparsity-Aware Split Finding

Sparsity = Data with many zeros or missing values

Common in:

- Text data (one-hot encoded)
- Sparse matrices
- High-dimensional data

XGBoost has a specialized mechanism to:

1. Detect sparsity
2. Assign default direction for sparse values
3. Optimize split finding

This is closely related to missing value handling.

6.4 Efficient Split Finding

The Problem with Exact Greedy Search

For continuous features, finding the best split requires testing many split points:

Feature: IQ values

[50, 55, 60, 65, 70, 75, 80, 85, 90, 95, 100, 105, ..., 150]



Potential splits:

52.5, 57.5, 62.5, 67.5, 72.5, 77.5, 82.5, ...

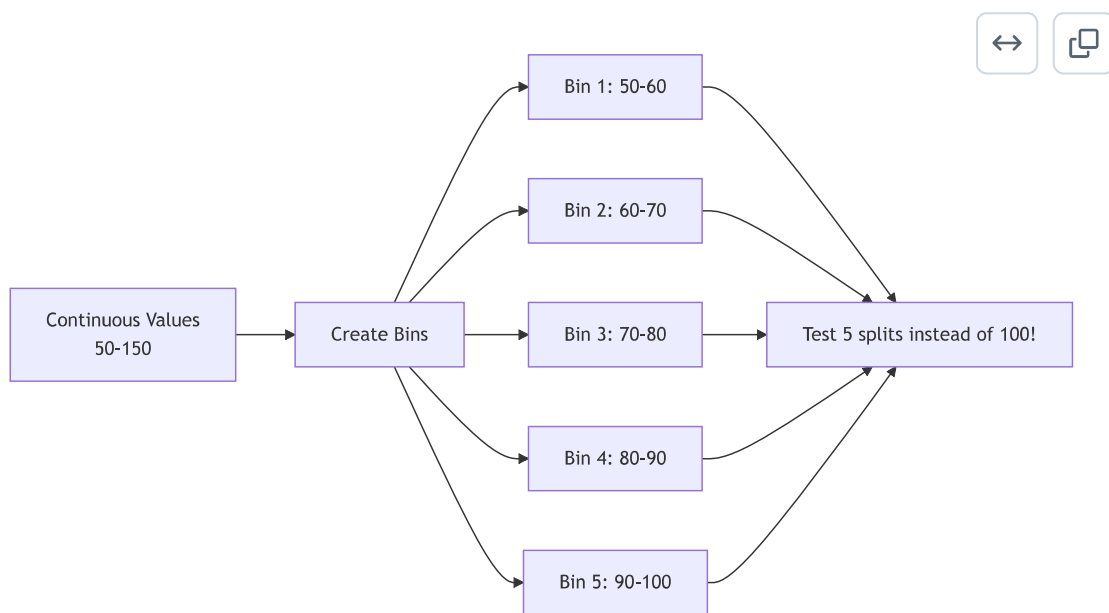
(n-1 split points for n values)

For large datasets:

- 1 million rows → ~1 million split evaluations
- 200 features → 200 million operations per node
- **Extremely slow!**

Solution 1: Approximate Tree Learning (Histogram Method)

Concept: Instead of testing every value, create **bins** (buckets).



Trade-off:

- **Exact Greedy:** Best accuracy, very slow
- **Histogram:** Slightly lower accuracy, much faster

Solution 2: Weighted Quantile Sketch

Problem with uniform binning:

Uniform bins: 0-10, 10-20, 20-30, ..., 90-100

But data distribution:

[Values 0-10: 50 points]

[Values 10-20: 5 points]

[Values 20-30: 5 points]

...

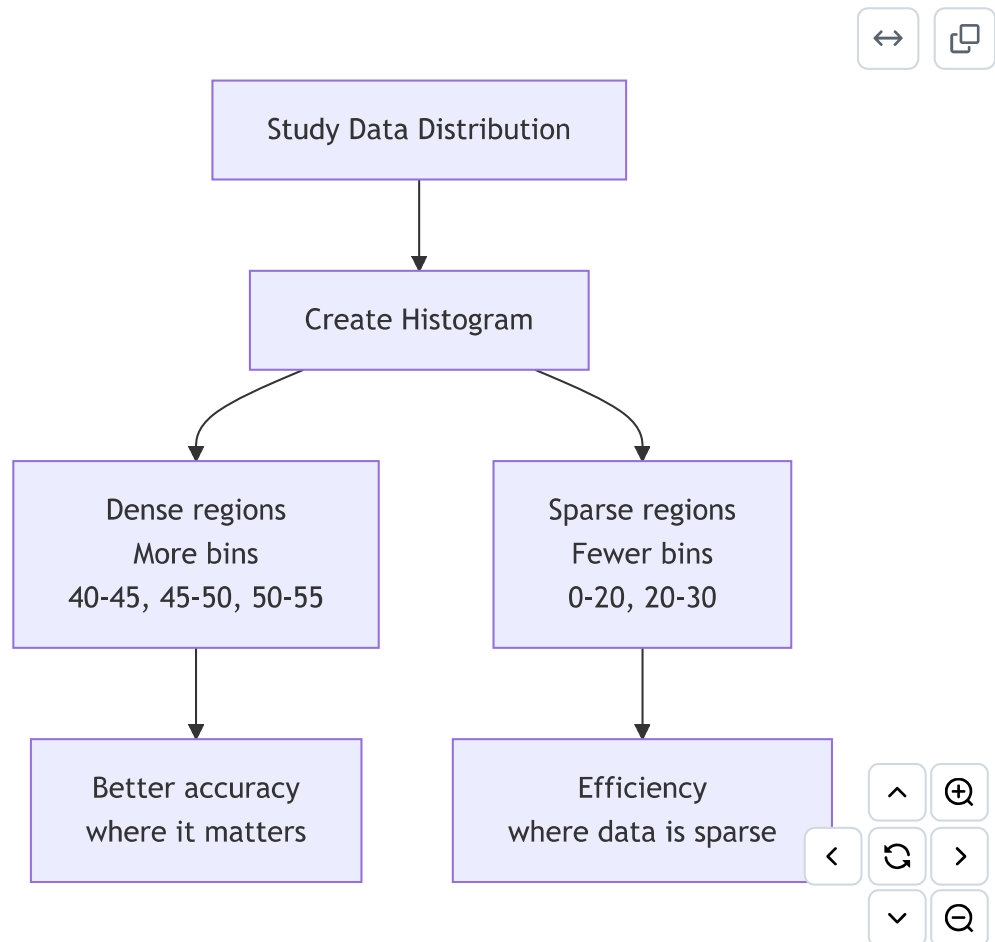
[Values 40-60: 500 points] ← Most data here!

...

[Values 90-100: 10 points]

Uniform bins don't respect data distribution!

Solution: Quantile-based binning



Quantile approach:

Create bins at quantiles: $Q_{0.2}, Q_{0.4}, Q_{0.6}, Q_{0.8}$

This ensures:

- Each bin contains similar number of samples
- Bins adapt to data distribution
- Better split quality

Example:

Data: [1, 2, 3, 45, 46, 47, 48, 49, 50, 100]



Uniform bins:

[1-25]: 3 points
[25-50]: 6 points
[50-75]: 0 points
[75-100]: 1 point

Quantile bins (4 bins):

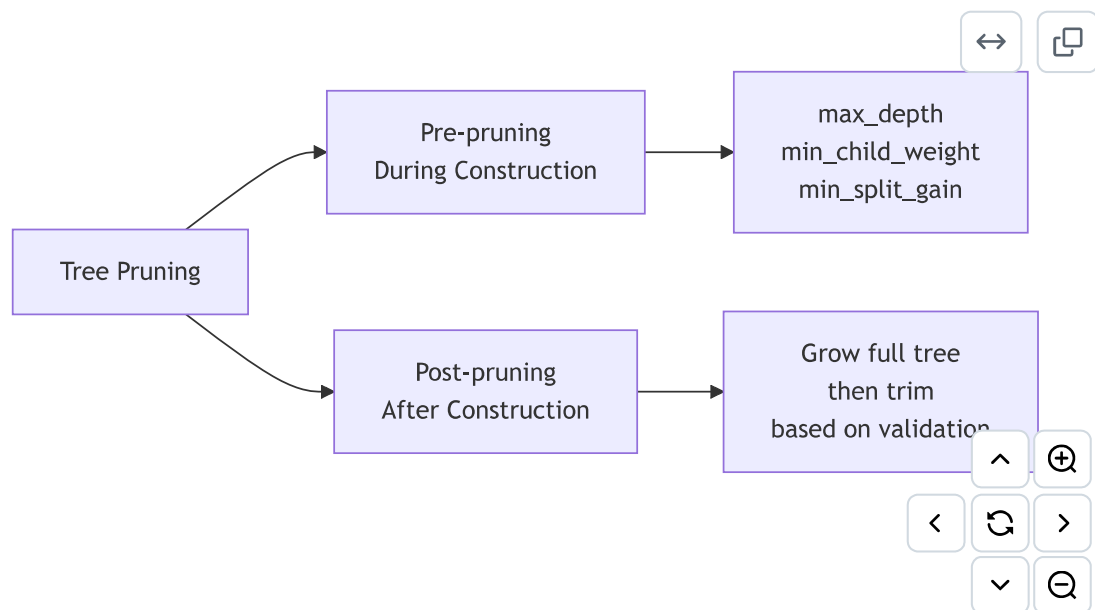
[1-3]: 3 points (25% quantile)
[3-47]: 3 points (50% quantile)
[47-49]: 2 points (75% quantile)
[49-100]: 2 points (100% quantile)

Benefits Summary

Method	Accuracy	Speed	Best For
Exact Greedy	Highest	Slowest	Small datasets
Uniform Histogram	Lower	Fast	Quick approximations
Quantile Histogram	Medium-High	Fast	Large datasets (XGBoost default)

6.5 Tree Pruning

Types of Pruning



XGBoost Pruning Parameters

Parameter	Type	Description
max_depth	Pre	Maximum tree depth
min_child_weight	Pre	Minimum sum of instance weight in child
gamma	Pre	Minimum loss reduction for split
alpha	Post	L1 regularization on leaf weights
lambda	Post	L2 regularization on leaf weights

Gamma Parameter (Special Feature)

The `gamma` parameter implements a **cost-complexity pruning**:

Split only if: $Gain > \gamma$

Where gain is:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

- G_L, G_R = Sum of gradients in left and right child
- H_L, H_R = Sum of Hessians in left and right child

Effect: Only create branches that significantly reduce loss.

Example

```
params = {  
    'max_depth': 5,           # Pre-pruning: limit depth  
    'min_child_weight': 1,    # Pre-pruning: min samples  
    'gamma': 0.1,            # Pre-pruning: min gain  
    'lambda': 1.0,           # Post-pruning: L2 reg  
    'alpha': 0.0,            # Post-pruning: L1 reg  
}
```



Result: Trees are more generalized and less prone to overfitting.

7. Comparison with Other Libraries {#comparison}

7.1 LightGBM

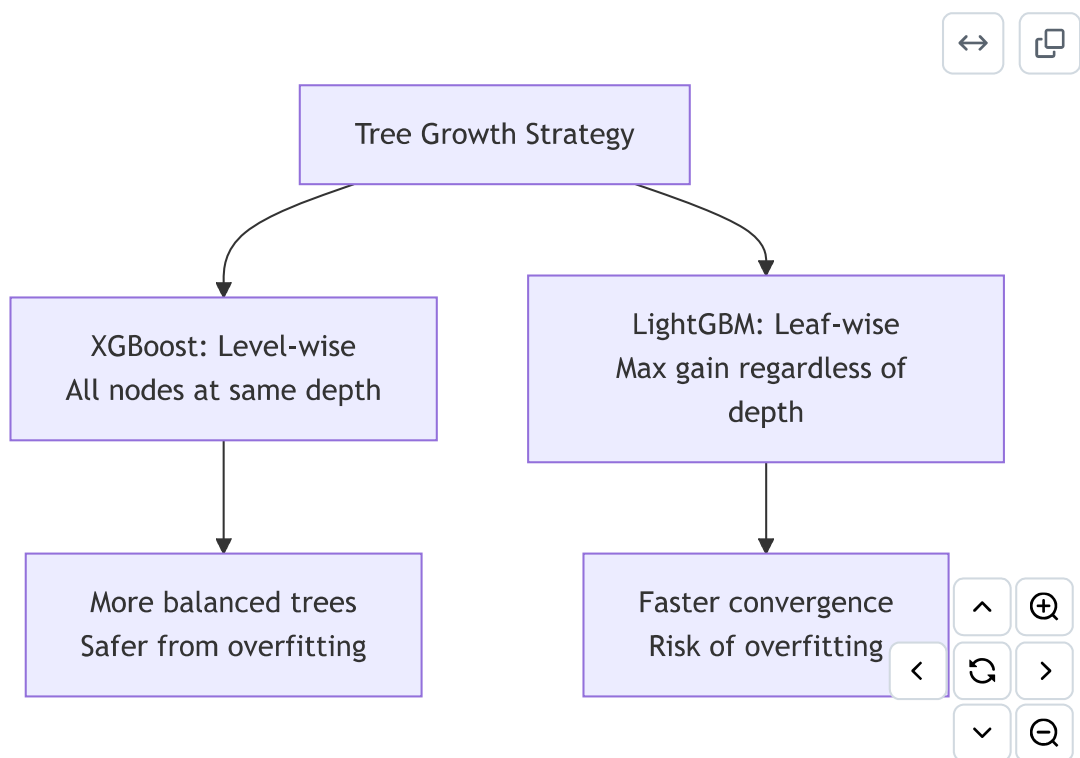
Developer: Microsoft Research

Key Features:

- Leaf-wise tree growth (vs level-wise in XGBoost)
- Gradient-based One-Side Sampling (GOSS)
- Exclusive Feature Bundling (EFB)

Comparison:

Aspect	XGBoost	LightGBM
Speed	Fast	Faster (especially large data)
Memory	Moderate	Lower memory usage
Accuracy	High	Comparable to XGBoost
Tree Growth	Level-wise	Leaf-wise
Best For	General purpose	Large datasets, limited memory



7.2 CatBoost

Developer: Yandex

Key Feature: Built-in support for **categorical features**

Comparison:

Aspect	XGBoost	CatBoost
Categorical Features	Manual encoding needed	Automatic handling
Speed	Fast	Fast (comparable)
Overfitting	Good with tuning	Excellent (built-in protection)
Target Encoding	Manual	Automatic ordered boosting
Best For	Numerical data	Mixed data with categories

Categorical Feature Handling Example

XGBoost:

```
# Manual encoding required
from sklearn.preprocessing import LabelEncoder

X['category'] = LabelEncoder().fit_transform(X['category'])

model = xgb.XGBClassifier()
model.fit(X, y)
```



CatBoost:

```
import catboost as cb

# Automatic handling
model = cb.CatBoostClassifier(cat_features=['category'])
model.fit(X, y) # No preprocessing needed!
```



7.3 Summary Table

Library	Best For	Key Strength	Main Limitation
XGBoost	General ML tasks, competitions	Most mature, well-documented	Slower than LightGBM on huge data
LightGBM	Large datasets, memory constraints	Speed, memory efficiency	Can overfit easier (leaf-wise)
CatBoost	Categorical data, minimal tuning	Automatic categorical handling	Less community resources

Recommendation: Start with XGBoost, experiment with others based on specific needs.

8. Mathematical Foundations {#mathematics}

8.1 Core Objective Function

The complete XGBoost objective:

$$L(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Where:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

8.2 Second-Order Approximation

XGBoost uses **second-order Taylor expansion** for optimization:

$$L^{(t)} \approx \sum_{i=1}^n \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

Where:

- $g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^{(t-1)}}$ (first derivative)
- $h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial (\hat{y}_i^{(t-1)})^2}$ (second derivative)

8.3 Optimal Weight Calculation

For a given tree structure, optimal leaf weight:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

8.4 Split Finding Criterion

The gain from splitting a node:

$$Gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

Where:

- $G_L = \sum_{i \in I_L} g_i, G_R = \sum_{i \in I_R} g_i$
- $H_L = \sum_{i \in I_L} h_i, H_R = \sum_{i \in I_R} h_i$

Split occurs only if: $Gain > 0$



Key Takeaways

1. **XGBoost = Gradient Boosting + Extreme Software Optimization**
2. **Three Core Focus Areas:**
 - Flexibility (cross-platform, multi-language, all ML problems)
 - Speed (6× optimizations: parallel, data structures, cache, out-of-core, distributed, GPU)
 - Performance (5× enhancements: regularization, missing values, sparsity, split finding, pruning)
3. **Dominance:** From 2016 onwards, XGBoost became the default choice for Kaggle competitions
4. **Alternatives:** LightGBM (speed), CatBoost (categorical features)
5. **Real Impact:** Training time reduced from hours to minutes on large datasets



References

- Original XGBoost Paper: "XGBoost: A Scalable Tree Boosting System" by Tianqi Chen and Carlos Guestrin
- Official Documentation: <https://xgboost.readthedocs.io/>

XGBoost for Regression: Complete Step-by-Step Guide

Overview

 Welcome to CampusX !! In this tutorial, we'll unravel the mathematical intricacies of XGBoost for regression, guiding you through each step of the process. Whether you're a data scientist, a machine learning enthusiast, or someone eager to dive into the mathematical foundations of algorithms, this video is tailor-made for you.

 Step-by-Step Guide :

Introduction to XGBoost Regression : Understand the fundamentals of XGBoost and its adaptability to regression problems.

Objective Function for Regression : Dive into the mathematical formulation of the objective function, exploring how XGBoost minimizes the difference between predicted and actual values.

Gradient Boosting Algorithm : Follow the step-by-step process of gradient boosting, where weak learners are sequentially added to the model to correct errors.

Regularization Techniques : Explore how XGBoost incorporates regularization terms in the objective function to prevent overfitting and enhance model generalization.

Tree Building Process : Break down the construction of individual decision trees and their role in improving the model's predictive power.

Prediction Calculation : Learn how XGBoost calculates predictions for regression tasks, combining the outputs of multiple weak learners.

 Hands-On Mathematical Calculation : Witness a practical demonstration where we apply the mathematical concepts to a sample dataset, showcasing how XGBoost refines its predictions through each iteration.

Problem Statement

Dataset

We have a small dataset with 4 students:

CGPA	Package (LPA)
6.7	4.5
9.0	11.0
7.5	6.0
5.0	8.0

Goal: Predict the package (salary in LPA - Lakhs Per Annum) based on CGPA using XGBoost.

Visualization

The scatter plot shows a non-linear relationship between CGPA (x-axis) and Package (y-axis).

How XGBoost Solves Regression Problems

Key Concept

XGBoost is essentially **Gradient Boosting** with optimizations for performance and speed. The core approach:

1. Start with a base estimator (usually the mean)
2. Build decision trees sequentially in stages
3. Each new tree predicts the residuals (errors) from the previous stage

4. Add trees with a learning rate multiplier
5. Continue until residuals approach zero or reach max estimators

Major Difference from Standard Gradient Boosting: XGBoost uses a different type of decision tree with a unique splitting criterion based on **Similarity Score** instead of Gini Impurity or Entropy.

Stage 1: Base Model (Mean)

Step 1.1: Calculate Mean of Target Variable

$$\text{Mean} = \frac{4.5 + 11.0 + 6.0 + 8.0}{4} = \frac{29.5}{4} = 7.375 \approx 7.3$$

Model 1 Predictions

For **all** data points, regardless of CGPA:

- Prediction = 7.3 LPA

Step 1.2: Calculate Residuals (Errors)

CGPA	Actual Package	Model 1 Prediction	Residual 1
6.7	4.5	7.3	-2.8
9.0	11.0	7.3	3.7
7.5	6.0	7.3	-1.3
5.0	8.0	7.3	0.7

Formula: $\text{Residual}_i = \text{Actual}_i - \text{Prediction}_i$

Observations:

- Positive residuals: Model underpredicted
- Negative residuals: Model overpredicted
- Goal: Reduce these residuals toward zero

Stage 2: Building the First Decision Tree

Overview

Build a decision tree to predict **Residual 1** using **CGPA** as the feature.

Key Formulas

Similarity Score

For any node containing residuals:

$$\text{Similarity Score (SS)} = \frac{(\sum \text{residuals})^2}{\text{number of residuals} + \lambda}$$

Where:

- λ = regularization parameter (we use $\lambda = 0$ for this example)

Gain

When splitting a node into left and right children:

$$\text{Gain} = SS_{\text{left}} + SS_{\text{right}} - SS_{\text{root}}$$

The split with the **highest gain** is chosen.

Output Value for Leaf Nodes

$$\text{Output} = \frac{\sum \text{residuals}}{\text{number of residuals} + \lambda}$$

When $\lambda = 0$, this simplifies to the **average of residuals** in that leaf.

Step 2.1: Initial Similarity Score (Root Node)

All residuals together: $[-2.8, 3.7, -1.3, 0.7]$

$$SS_{\text{root}} = \frac{(-2.8 + 3.7 - 1.3 + 0.7)^2}{4 + 0} = \frac{(0.3)^2}{4} = \frac{0.09}{4} = 0.02$$

Step 2.2: Find Best Split - Determine Splitting Criteria

Sort CGPA values: $[5.0, 6.7, 7.5, 9.0]$

Potential split points (midpoints between consecutive values):

Split Point	Between CGPAs
5.85	5.0 and 6.7
7.1	6.7 and 7.5
8.25	7.5 and 9.0

Step 2.3: Evaluate Split 1 - CGPA < 5.85

Split:

- Left: CGPA < 5.85 → [5.0] → Residual: [0.7]
- Right: CGPA ≥ 5.85 → [6.7, 7.5, 9.0] → Residuals: [-2.8, -1.3, 3.7]

Calculations:

$$SS_{\text{left}} = \frac{(0.7)^2}{1 + 0} = \frac{0.49}{1} = 0.49$$

$$SS_{\text{right}} = \frac{(-2.8 - 1.3 + 3.7)^2}{3 + 0} = \frac{(-0.4)^2}{3} = \frac{0.16}{3} = 0.05$$

$$\text{Gain} = 0.49 + 0.05 - 0.02 = 0.52$$

Step 2.4: Evaluate Split 2 - CGPA < 7.1

Split:

- Left: CGPA < 7.1 → [5.0, 6.7] → Residuals: [0.7, -2.8]
- Right: CGPA ≥ 7.1 → [7.5, 9.0] → Residuals: [-1.3, 3.7]

Calculations:

$$SS_{\text{left}} = \frac{(0.7 - 2.8)^2}{2 + 0} = \frac{(-2.1)^2}{2} = \frac{4.41}{2} = 2.20$$

$$SS_{\text{right}} = \frac{(-1.3 + 3.7)^2}{2 + 0} = \frac{(2.4)^2}{2} = \frac{5.76}{2} = 2.88$$

$$\text{Gain} = 2.20 + 2.88 - 0.02 = 5.06$$

Step 2.5: Evaluate Split 3 - CGPA < 8.25

Split:

- Left: $CGPA < 8.25 \rightarrow [5.0, 6.7, 7.5] \rightarrow \text{Residuals: } [0.7, -2.8, -1.3]$
- Right: $CGPA \geq 8.25 \rightarrow [9.0] \rightarrow \text{Residuals: } [3.7]$

Calculations:

$$SS_{\text{left}} = \frac{(0.7 - 2.8 - 1.3)^2}{3 + 0} = \frac{(-3.4)^2}{3} = \frac{11.56}{3} = 3.85$$

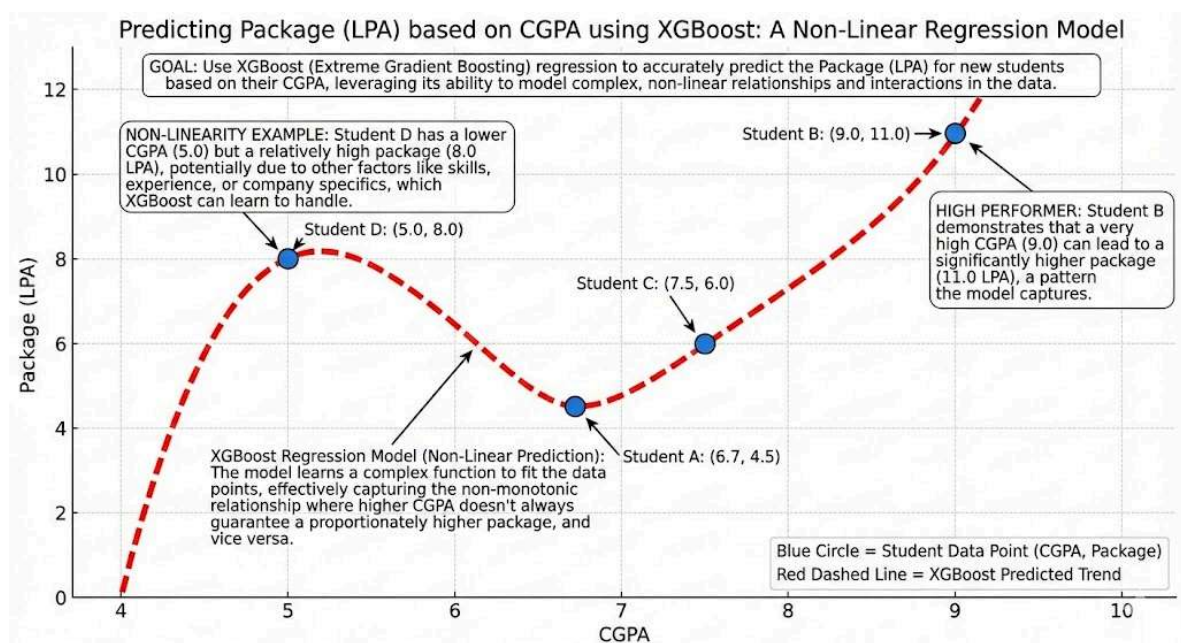
$$SS_{\text{right}} = \frac{(3.7)^2}{1 + 0} = \frac{13.69}{1} = 13.69$$

$$\text{Gain} = 3.85 + 13.69 - 0.02 = 17.52$$

Step 2.6: Select Best Root Split

Split Criteria	Gain
$CGPA < 5.85$	0.52
$CGPA < 7.1$	5.06
$CGPA < 8.25$	17.52 ✓

Winner: $CGPA < 8.25$ (highest gain)



Step 2.7: Further Split the Left Branch

The right branch has only one residual, so it becomes a leaf. Now split the left branch with residuals: [0.7, -2.8, -1.3]

Possible splits:

- CGPA < 5.85
- CGPA < 7.1

Split Option 1: CGPA < 5.85

- Left: [0.7]
- Right: [-2.8, -1.3]

$$SS_L = \frac{(0.7)^2}{1} = 0.49$$

$$SS_R = \frac{(-2.8 - 1.3)^2}{2} = \frac{(-4.1)^2}{2} = \frac{16.81}{2} = 8.40$$

$$\text{Gain} = 0.49 + 8.40 - 3.85 = 5.04$$

Split Option 2: CGPA < 7.1

- Left: [0.7, -2.8]
- Right: [-1.3]

$$SS_L = \frac{(0.7 - 2.8)^2}{2} = \frac{(-2.1)^2}{2} = \frac{4.41}{2} = 2.20$$

$$SS_R = \frac{(-1.3)^2}{1} = 1.69$$

$$\text{Gain} = 2.20 + 1.69 - 3.85 = 0.04$$

Winner: CGPA < 5.85 (gain = 5.04 > 0.04)

Step 2.8: Final Decision Tree Structure

Step 2.9: Calculate Leaf Output Values

Using the formula (with $\lambda = 0$):

$$\text{Output} = \frac{\sum \text{residuals}}{\text{number of residuals}}$$

Leaf	Residuals	Calculation	Output
Right (CGPA ≥ 8.25)	[3.7]	$\frac{3.7}{1}$	3.7
Left-Right ($5.85 \leq \text{CGPA} < 8.25$)	[-2.8, -1.3]	$\frac{-2.8 - 1.3}{2} = \frac{-4.1}{2}$	-2.05
Left-Left (CGPA < 5.85)	[0.7]	$\frac{0.7}{1}$	0.7

Stage 2: Combined Model

Model Formula

$$\text{Model}_2 = \text{Mean} + \eta \times \text{DecisionTree}_1$$

Where:

- Mean = 7.3
- η (eta) = learning rate = 0.3 (default in XGBoost)
- DecisionTree_1 = output from first tree

Predictions for Each Data Point

For each student, we:

1. Start with mean: 7.3
2. Traverse the tree based on CGPA
3. Add: $0.3 \times \text{tree output}$

Student 1: CGPA = 6.7

Tree traversal:

- CGPA < 8.25 ? **Yes** → go left
- CGPA < 5.85 ? **No** → go right
- **Leaf output:** -2.05

$$\text{Prediction} = 7.3 + 0.3 \times (-2.05) = 7.3 - 0.615 = 6.69$$

Student 2: CGPA = 9.0

Tree traversal:

- CGPA < 8.25 ? **No** → go right

- Leaf output: 3.7

$$\text{Prediction} = 7.3 + 0.3 \times 3.7 = 7.3 + 1.11 = 8.41$$

Student 3: CGPA = 7.5

Tree traversal:

- CGPA < 8.25? **Yes** → go left
- CGPA < 5.85? **No** → go right
- Leaf output: -2.05

$$\text{Prediction} = 7.3 + 0.3 \times (-2.05) = 6.69$$

Student 4: CGPA = 5.0

Tree traversal:

- CGPA < 8.25? **Yes** → go left
- CGPA < 5.85? **Yes** → go left
- Leaf output: 0.7

$$\text{Prediction} = 7.3 + 0.3 \times 0.7 = 7.3 + 0.21 = 7.51$$

Step 2.10: Calculate New Residuals (Residual 2)

CGPA	Actual	Model 1	Residual 1	Model 2	Residual 2
6.7	4.5	7.3	-2.8	6.69	-2.19
9.0	11.0	7.3	3.7	8.41	2.59
7.5	6.0	7.3	-1.3	6.69	-0.69
5.0	8.0	7.3	0.7	7.51	0.49

Observation: All residuals have moved closer to zero! ✓

- $|-2.8| > |-2.19|$
- $|3.7| > |2.59|$
- $|-1.3| > |-0.69|$
- $|0.7| > |0.49|$

This confirms Model 2 is performing better than Model 1.

Stage 3 and Beyond

Continue the Process

For Stage 3, we would:

1. Build another decision tree using CGPA and **Residual 2**
2. Find the best splits using similarity scores and gain
3. Create a new combined model:

$$\text{Model}_3 = \text{Mean} + \eta \times DT_1 + \eta \times DT_2$$

4. Calculate Residual 3
5. Repeat...

Stopping Criteria

Continue adding trees until:

- Residuals approach zero
- Reach maximum number of estimators (e.g., 100, 200)
- No significant improvement in error

General Formula

$$\text{Final Model} = \text{Mean} + \eta \sum_{i=1}^n \text{DecisionTree}_i$$

Where n = number of estimators (trees)

Key Differences: XGBoost vs Standard Decision Trees

Aspect	Standard Decision Tree	XGBoost Decision Tree
Splitting Criterion	Gini Impurity or Entropy	Similarity Score (gain-based)
Leaf Output	Mean/Mode of samples	Formula: $\frac{\sum \text{residuals}}{n + \lambda}$
Purpose	Direct prediction	Predict residuals sequentially
Regularization	Limited	Built-in with λ parameter

Important Formulas Summary

1. Similarity Score

$$SS = \frac{\left(\sum_{i=1}^n r_i \right)^2}{n + \lambda}$$

Where:

- r_i = residual values in the node
- n = number of residuals
- λ = regularization parameter (L2)

2. Gain from Split

$$\text{Gain} = SS_{\text{left}} + SS_{\text{right}} - SS_{\text{parent}}$$

3. Leaf Output Value

$$\text{Output} = \frac{\sum_{i=1}^n r_i}{n + \lambda}$$

4. Combined Model Prediction

$$\hat{y} = \text{base_prediction} + \eta \sum_{t=1}^T f_t(x)$$

Where:

- η = learning rate (typically 0.3)
- T = number of trees
- $f_t(x)$ = output from tree t

Hyperparameters Mentioned

Parameter	Symbol	Default Value	Purpose
Learning Rate	η	0.3	Shrinks contribution of each tree (regularization)

Parameter	Symbol	Default Value	Purpose
Lambda	λ	0 (in our example)	L2 regularization on leaf weights
Max Depth	-	6	Maximum depth of trees
Number of Estimators	n	100	Total trees to build

Practice Tasks

Task 1: Build Decision Tree 2

Using the Stage 2 residuals:

CGPA	Residual 2
6.7	-2.19
9.0	2.59
7.5	-0.69
5.0	0.49

Build the second decision tree following the same steps:

1. Calculate initial similarity score
2. Find all splitting criteria
3. Evaluate each split
4. Select the best splits
5. Calculate leaf outputs
6. Make predictions and calculate Residual 3

Task 2: Multiple Features

Question: How does tree construction change when you have multiple features?

Example: CGPA + 12th Grade Marks → Package

Hint: You'll need to:

- Consider split points for each feature separately

- Compare gains across all features
- Select the feature and split point with highest gain

Task 3: Categorical Features

Question: How to handle categorical features?

Cases to consider:

1. **Binary categorical** (e.g., Gender: Male/Female)
2. **Multi-class categorical** (e.g., Branch: CS/EC/ME/CE)

Hint: Research one-hot encoding and how XGBoost handles categorical splits natively.

Unanswered Questions (Covered in Future Videos)

1. Why these specific formulas?

- Where does the similarity score formula come from?
- Why is the leaf output formula structured this way?
- **Answer:** Mathematical derivation from the optimization objective (covered in "Mathematical Formulation" video)

2. What is Lambda (λ)?

- How does it prevent overfitting?
- How to tune it?
- **Answer:** Covered in "Regularization in XGBoost" video

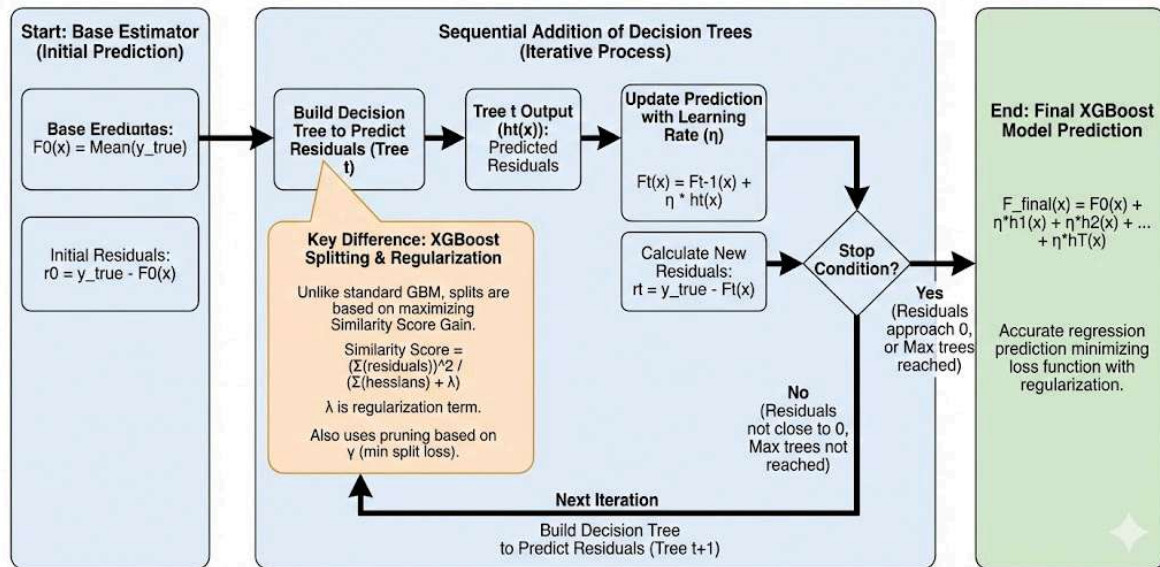
3. Approximate Algorithm

- For large datasets, exact greedy algorithm is slow
- How does the approximate method work?
- **Answer:** Covered in dedicated video on "Approximate Split Finding"

4. Tree Construction Methods

- Exact Greedy Algorithm (what we learned)
- Approximate Algorithm (histogram-based)

High-Level XGBoost Regression Process: Sequential Residual Fitting & Similarity-Based Splitting



Tree Construction: Exact Greedy Algorithm

Algorithm Overview

When to Use

- **Small datasets:** Exact greedy algorithm (tries all possible splits)
- **Large datasets:** Approximate algorithm (uses quantiles/histograms)

Residual Reduction Pattern

Stage	Student 1	Student 2	Student 3	Student 4
Residual 1	-2.8	3.7	-1.3	0.7
Residual 2	-2.19 ✓	2.59 ✓	-0.69 ✓	0.49 ✓
Goal	→ 0	→ 0	→ 0	→ 0

All residuals are moving toward zero, indicating model improvement!

Conclusion

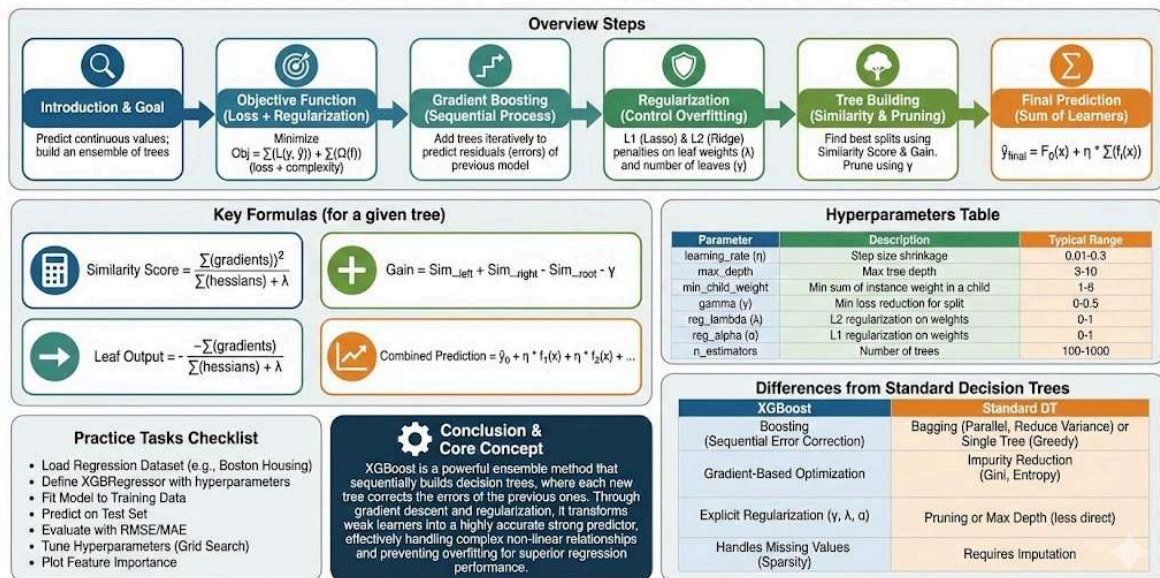
XGBoost regression works by:

1. Starting with a simple base model (mean)
2. Sequentially adding decision trees that predict residuals
3. Using similarity score for optimal splits
4. Combining predictions with learning rate
5. Iteratively reducing errors toward zero

The "magic" lies in:

- Custom splitting criterion (similarity score)
- Regularization built into formulas
- Sequential error correction
- Ensemble of weak learners → strong predictor

XGBoost for Regression: From Weak Learners to a Strong Predictor



XGBoost for Classification - Detailed Notes

Prerequisites

Explore how XGBoost takes on classification problems with precision and efficiency. From its intuitive framework to tackling real-world datasets, this video breaks down the essentials of using XGBoost for classification tasks. In this tutorial, we'll unravel the mathematical intricacies of XGBoost for classification, guiding you through each step of the process.

1. **XGBoost for Regression** - Understanding the basic framework
2. **Gradient Boosting for Classification** - How gradient boosting works in classification scenarios
3. **Log Odds (Log of Odds)** - A fundamental concept used in logistic regression and boosting

Note: XGBoost = Extreme Gradient Boosting. The "Extreme" comes from various optimizations that improve both performance and speed.

Problem Statement

We'll work with a simple toy dataset to understand XGBoost classification:

CGPA	Placed?
5.70	0
6.25	1
7.10	0
8.15	1
9.60	1

Goal: Build an XGBoost classification model that predicts whether a student will be placed (1) or not (0) based on their CGPA.

Overview - How XGBoost Solves Classification

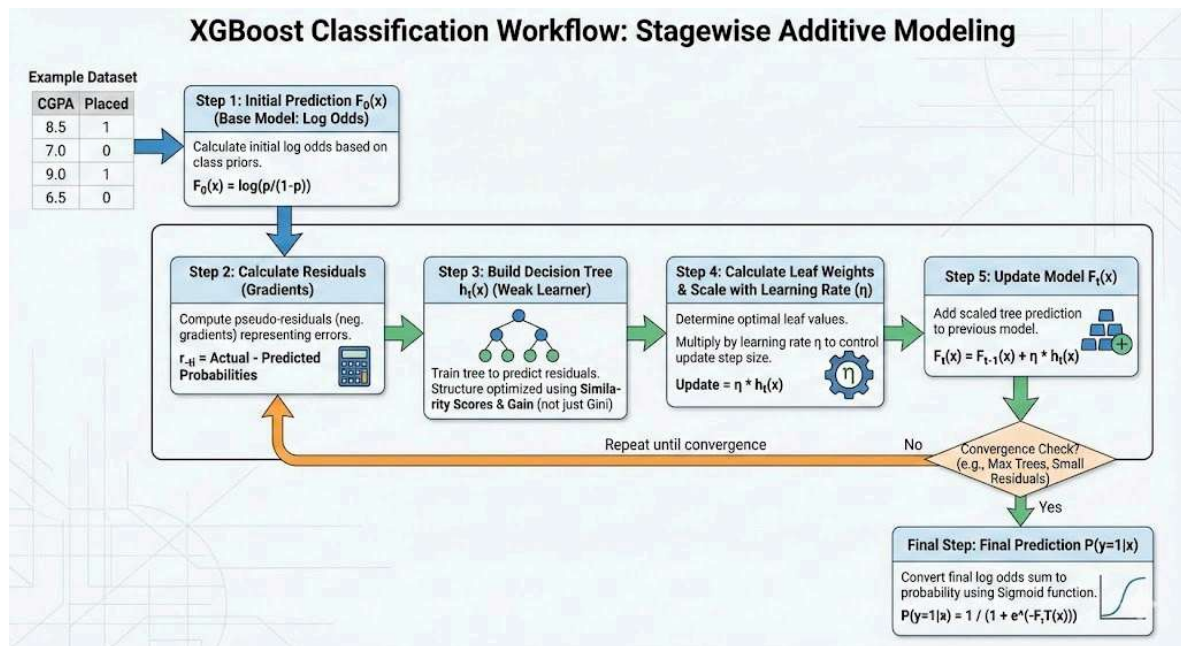
XGBoost follows the same workflow as Gradient Boosting, using **stagewise additive modeling**:

Key Workflow Steps:

1. Start with a **base model** (log of odds)
2. Calculate errors (residuals)
3. Build a **decision tree** on those errors
4. Add the tree to the model (with learning rate)
5. Repeat until errors approach zero

Main Difference from Gradient Boosting:

XGBoost uses different decision trees - instead of using entropy or Gini index, XGBoost uses **Similarity Score** as the splitting criterion.



Step-by-Step Solution

Stage 1: Base Estimator (Log of Odds)

In regression, we start with the mean. In classification, we start with **log of odds**.

Log of Odds Formula:

$$\log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

where p = probability of positive class (placed = 1)

Calculation:

$$p = \frac{\text{Number of 1s}}{\text{Total}} = \frac{3}{5}$$

$$\log(\text{odds}) = \log\left(\frac{3/5}{2/5}\right) = \log\left(\frac{3}{2}\right) = 0.405$$

So our **Stage 1 prediction** for every data point is **0.405** (in log odds space).

Converting to Probability:

$$P = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}} = \frac{e^{0.405}}{1 + e^{0.405}} = 0.6$$

Interpretation: The model predicts 60% probability of placement for everyone, regardless of CGPA. This is clearly not accurate!

Stage 1 Results:

CGPA	Placed?	Pred1(log odds)	Pred1(prob)	Residual 1
5.70	0	0.405	0.6	-0.6
6.25	1	0.405	0.6	0.4
7.10	0	0.405	0.6	-0.6
8.15	1	0.405	0.6	0.4
9.60	1	0.405	0.6	0.4

Residual = Actual - Predicted Probability

Stage 2: Building the First Decision Tree

Now we build a decision tree to predict the residuals using CGPA as input.

Step 1: Create Root Node with All Residuals

Put all residuals in one leaf node:

Leaf Node: [-0.6, 0.4, -0.6, 0.4, 0.4]



Step 2: Calculate Similarity Score

For classification, the **Similarity Score** formula is:

$$SS = \frac{(\sum \text{residual}_i)^2}{\sum [\text{prev_prob}_i \times (1 - \text{prev_prob}_i)] + \lambda}$$

For root node:

$$SS_{\text{root}} = \frac{(-0.6 + 0.4 + (-0.6) + 0.4 + 0.4)^2}{5 \times (0.6 \times 0.4) + 0}$$

$$= \frac{(0)^2}{5 \times 0.24} = \frac{0}{1.2} = 0$$

Step 3: Find Best Split

We need to try different splitting criteria based on CGPA. Splitting criteria are averages of consecutive values:

Split Criteria	Calculation
5.97	$(5.70 + 6.25)/2$
6.67	$(6.25 + 7.10)/2$
7.62	$(7.10 + 8.15)/2$
8.87	$(8.15 + 9.60)/2$

Testing Split: CGPA < 5.97

Left Leaf (CGPA < 5.97):

$$SS_L = \frac{(-0.6)^2}{1 \times (0.6 \times 0.4)} = \frac{0.36}{0.24} = 1.5$$

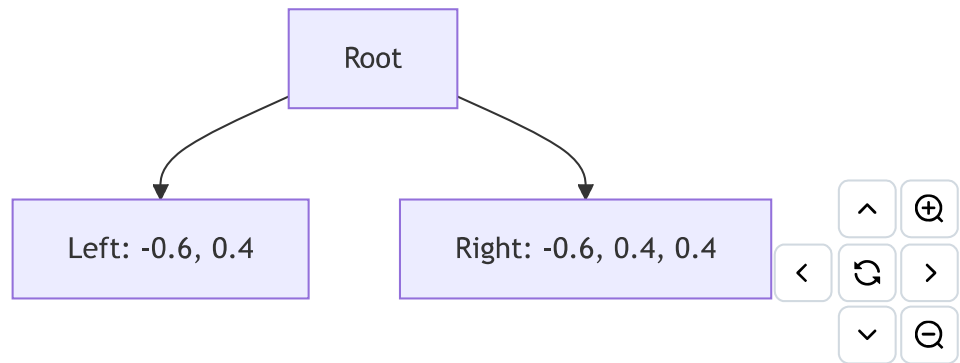
Right Leaf (CGPA ≥ 5.97):

$$SS_R = \frac{(0.4 + (-0.6) + 0.4 + 0.4)^2}{4 \times (0.6 \times 0.4)} = \frac{(0.6)^2}{0.96} = \frac{0.36}{0.96} = 0.375$$

Gain:

$$\text{Gain} = SS_L + SS_R - SS_{\text{root}} = 1.5 + 0.375 - 0 = 1.875$$

Testing Split: CGPA < 6.67



Left Leaf:

$$SS_L = \frac{(-0.6 + 0.4)^2}{2 \times 0.24} = \frac{0.04}{0.48} = 0.083$$

Right Leaf:

$$SS_R = \frac{(-0.6 + 0.4 + 0.4)^2}{3 \times 0.24} = \frac{0.04}{0.72} = 0.056$$

Gain: $0.083 + 0.056 - 0 = 0.139$

Testing Split: CGPA < 7.62 ✓



Left Leaf:

$$SS_L = \frac{(-0.6 + 0.4 + (-0.6))^2}{3 \times 0.24} = \frac{(-0.8)^2}{0.72} = \frac{0.64}{0.72} = 0.889$$

Right Leaf:

$$SS_R = \frac{(0.4 + 0.4)^2}{2 \times 0.24} = \frac{(0.8)^2}{0.48} = \frac{0.64}{0.48} = 1.333$$

Gain: $0.889 + 1.333 - 0 = 2.222 \leftarrow \text{MAXIMUM!}$

Testing Split: CGPA < 8.87

This will produce lower gain than 2.222 (similar residuals get separated).

Winner: CGPA < 7.62 with Gain = 2.222

Stage 2: Calculate Output Values for Leaves

For each leaf node, we calculate the output value using:

$$\text{Output} = \frac{\sum \text{residual}_i}{\sum [\text{prev_prob}_i \times (1 - \text{prev_prob}_i)] + \lambda}$$

Left Leaf (CGPA < 7.62): [-0.6, 0.4, -0.6]

$$\text{Output}_L = \frac{-0.6 + 0.4 + (-0.6)}{3 \times (0.6 \times 0.4) + 0} = \frac{-0.8}{0.72} = -1.11$$

Right Leaf (CGPA ≥ 7.62): [0.4, 0.4]

$$\text{Output}_R = \frac{0.4 + 0.4}{2 \times (0.6 \times 0.4) + 0} = \frac{0.8}{0.48} = 1.66$$

Stage 2: Making Predictions

The combined model is:

$$\text{Pred2}(\log \text{ odds}) = \text{Pred1} + \eta \times \text{Tree Output}$$

where $\eta = 0.3$ (learning rate)

Prediction Calculations:

CGPA	Condition	Calculation	Pred2(log odds)
5.70	< 7.62	$0.405 + 0.3 \times (-1.11)$	0.072
6.25	< 7.62	$0.405 + 0.3 \times (-1.11)$	0.072
7.10	< 7.62	$0.405 + 0.3 \times (-1.11)$	0.072
8.15	≥ 7.62	$0.405 + 0.3 \times (1.66)$	0.903
9.60	≥ 7.62	$0.405 + 0.3 \times (1.66)$	0.903

Convert to Probabilities:

$$P = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

CGPA	Pred2(log odds)	Pred2(prob)
5.70	0.072	0.518
6.25	0.072	0.518
7.10	0.072	0.518
8.15	0.903	0.712
9.60	0.903	0.712

New Residuals:

CGPA	Actual	Pred2(prob)	Residual 2
5.70	0	0.518	-0.518
6.25	1	0.518	0.482
7.10	0	0.518	-0.518
8.15	1	0.712	0.288
9.60	1	0.712	0.288

Progress Check:

- Row 1: -0.6 → -0.518 (closer to 0) ✓
- Row 2: 0.4 → 0.482 (slightly worse)
- Row 3: -0.6 → -0.518 (closer to 0) ✓
- Row 4: 0.4 → 0.288 (closer to 0) ✓
- Row 5: 0.4 → 0.288 (closer to 0) ✓

Overall, the model is improving!

Stage 3 and Beyond

The process continues:

For Stage 3:

1. Take CGPA and Residual 2
2. Build another decision tree (Model 3)
3. Combine: $\text{Pred3} = 0.405 + 0.3 \times \text{Tree1} + 0.3 \times \text{Tree2}$

4. Calculate new residuals
5. Repeat...

Continue until residuals ≈ 0

Key Formulas Summary

1. Log of Odds (Base Prediction)

$$\log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

2. Converting Log Odds to Probability

$$P = \frac{e^{\log(\text{odds})}}{1 + e^{\log(\text{odds})}}$$

3. Similarity Score (for splitting)

$$SS = \frac{(\sum \text{residual}_i)^2}{\sum [\text{prev_prob}_i \times (1 - \text{prev_prob}_i)] + \lambda}$$

4. Gain (for choosing best split)

$$\text{Gain} = SS_{\text{left}} + SS_{\text{right}} - SS_{\text{root}}$$

5. Output Value for Leaf

$$\text{Output} = \frac{\sum \text{residual}_i}{\sum [\text{prev_prob}_i \times (1 - \text{prev_prob}_i)] + \lambda}$$

6. Final Prediction (stagewise)

$$\text{Pred}_n = \log(\text{odds}) + \eta \sum_{i=1}^{n-1} \text{Tree}_i$$

Comparison: XGBoost vs Gradient Boosting

Aspect	Gradient Boosting	XGBoost
Base approach	Stagewise additive modeling	Stagewise additive modeling
Starting point	$\log(\text{odds})$	$\log(\text{odds})$
Tree building	Entropy/Gini Index	Similarity Score
Output calculation	Similar formula	Similar formula (no square in numerator)
Overall workflow	✓ Same	✓ Same

Key Difference: The mathematical formulation for building trees (Similarity Score) and calculating outputs.

Why These Formulas?

The formulas for:

- Similarity Score
- Output values

come from mathematical optimization principles. These will be covered in the next video on **Mathematical Formulation of XGBoost**.

Summary

XGBoost for classification works by:

1. **Starting with $\log(\text{odds})$** as the base prediction
2. **Iteratively building decision trees** to predict residuals (errors)
3. **Using Similarity Score** to find optimal splits
4. **Combining models** with a learning rate
5. **Converting $\log(\text{odds})$ to probabilities** when needed

The workflow is identical to Gradient Boosting, but the tree-building mechanism (Similarity Score) is different, leading to better performance and speed.

Complete Example Summary Table

Stage	Model Components	First Prediction	Last Prediction
1	$\log(\text{odds}) = 0.405$	0.6 (prob)	0.6 (prob)
2	$M1 + 0.3 \times \text{Tree1}$	0.518	0.712
3	$M1 + 0.3 \times (\text{Tree1} + \text{Tree2})$	...	...
n	$M1 + 0.3 \times \sum \text{Trees}$	→ target	→ target

Goal: Residuals → 0 (perfect predictions)

